RetailMart
Daily Revenue Pipeline

End-to-End Airflow Project from Scratch

The hands-on project that makes Backfill · Catchup · Versioning · Promotion real

Astronomer Astro CLI | **SQLite Local DB** | **Python Operators** | **No External APIs**

Why You Need This Project First

The four Airflow operations — backfill, catchup, versioning, and environment promotion — are **meaningless without a real pipeline to practise on**. You cannot backfill a DAG that never ran. You cannot observe catchup behaviour if there is no historical gap. You cannot version a DAG with no logic to change. This project gives you exactly that foundation.

Operation	What This Project Makes You Do
Backfill	Revenue data from Jan 2026 was never loaded. Run backfill to fill the gap.
Catchup	Re-enable a paused DAG and watch 30 past intervals auto-schedule.
Versioning	A new discount column is added in v2. Migrate live without breaking history.
Promotion	Deploy dev→stage→prod using Astro GitHub Actions with the same DAG.

> **★ IMPORTANT**
>
> Complete this project first. Then every command in the Advanced Operations guide will have a real DAG and real data to operate on.

verview & Architecture

What we are building, why, and how all the pieces connect

1.1 The Business Problem

RetailMart is a fictional e-commerce company selling products across three categories: Electronics, Clothing, and Groceries. Every day, a transactional database records every order. The analytics team needs a **daily revenue summary** loaded into a reporting table by 6 AM so that the morning dashboard is fresh for the business review.

The pipeline was launched on **1 May 2026** but the business later discovered that **January–April 2026 data existed in the source system and was never loaded**. This creates the perfect backfill scenario.

1.2 What the Pipeline Does (DAG Logic)

#	Task ID	Operator	What It Does
1	generate_orders	Python	Simulates raw order CSV for the data interval date.
2	validate_orders	Python	Checks for nulls, negative amounts, unknown categories.
3	load_to_staging	Python	Inserts raw rows into SQLite staging table.
4	compute_revenue	Python	Aggregates revenue by category for the day.
5	load_to_report	Python	Upserts daily summary into the reporting table (idempotent).
6	audit_log	Python	Writes a run audit record (dag_id, date, row count, status).

1.3 Why Each Task Was Chosen

Every task was designed to demonstrate a specific Airflow principle:

generate_orders uses context['data_interval_start']

Forces the task to generate data for the correct historical date during backfill — not today.

load_to_report uses INSERT OR REPLACE

Makes the task idempotent: re-running the same date produces no duplicates.

audit_log records dag_id + version

When you introduce v2, the audit table shows exactly which version ran each date.

validate_orders can fail intentionally

Inject bad data for a date to practise --rerun-failed-tasks backfill.

etup from Zero

Initialise the Astro project, folder structure, and all supporting files

Step 1 — Initialise the Astro Project

If you already have an active Astro deployment you can work locally and push, or connect directly. These commands create the project skeleton.

```
# Create a new directory for the project
mkdir retailmart-pipeline && cd retailmart-pipeline
# Initialise an Astronomer Astro project
astro dev init
# This generates the standard Astro scaffold:
#   .astro/
#   dags/
#   include/
#   plugins/
#   tests/
#   Dockerfile
#   packages.txt
#   requirements.txt
#   .gitignore
# Start the local Airflow environment
astro dev start
# Confirm Airflow UI is up at http://localhost:8080
# Default login: admin / admin
```

Step 2 — Install Dependencies

Add required packages to requirements.txt:

```
# requirements.txt  (Astro Runtime already includes apache-airflow)
pandas==2.2.2
faker==25.0.0
```

> ■ NOTE
>
> After editing requirements.txt, restart the local environment: astro dev restart

Step 3 — Final Folder Structure

Create the files listed below. All paths are relative to the project root.

```
retailmart-pipeline/
■■■ dags/
■ ■■■ retailmart_revenue_v1.py # Main DAG — version 1
■ ■■■ retailmart_revenue_v2.py # Upgraded DAG — version 2 (added later)
■ ■■■ catchup_compare/
■ ■■■ catchup_true_demo.py # Catchup demo DAG A
```

```
■ ■■■ catchup_false_demo.py # Catchup demo DAG B
■■■ include/
■ ■■■ db_utils.py # SQLite helpers (shared across DAG versions)
■ ■■■ data_gen.py # Order data generator
■■■ tests/
■ ■■■ test_retailmart_dag.py # Pytest DAG integrity tests
■■■ .github/
■ ■■■ workflows/
■ ■■■ astro_deploy.yml # CI/CD pipeline
■■■ requirements.txt
■■■ Dockerfile
```

Source Files

Every file you need, fully commented, copy-paste ready

File 1 — include/db_utils.py

Shared SQLite helper used by every task in both v1 and v2. Using SQLite means **zero external database setup** — the DB lives inside the container at /tmp/retailmart.db.

```
# include/db_utils.py
"""
db_utils.py — SQLite database helpers for RetailMart pipeline.
All tables are created idempotently (CREATE TABLE IF NOT EXISTS).
"""
import sqlite3, os
DB_PATH = os.environ.get('RETAILMART_DB', '/tmp/retailmart.db')
def get_conn():
    """Return a SQLite connection; creates the DB file if absent."""
    return sqlite3.connect(DB_PATH)
def init_db():
    """Create all tables. Safe to call on every DAG run (idempotent)."""
    conn = get_conn()
    cur  = conn.cursor()
    # Raw orders — staging table
    cur.execute('''
        CREATE TABLE IF NOT EXISTS raw_orders (
            order_id    TEXT PRIMARY KEY,
            order_date  TEXT NOT NULL,
            category    TEXT NOT NULL,
            amount      REAL NOT NULL,
            customer_id TEXT NOT NULL
        )
    ''')
    # Daily revenue report table (v1 schema)
    cur.execute('''
        CREATE TABLE IF NOT EXISTS daily_revenue (
            report_date TEXT NOT NULL,
            category    TEXT NOT NULL,
            total_rev   REAL NOT NULL,
            order_count INTEGER NOT NULL,
            dag_version TEXT NOT NULL DEFAULT 'v1',
            loaded_at   TEXT NOT NULL,
            PRIMARY KEY (report_date, category)
        )
    ''')
    # Audit log — one row per DAG run
    cur.execute('''
        CREATE TABLE IF NOT EXISTS pipeline_audit (
            run_id      TEXT PRIMARY KEY,
            dag_id      TEXT NOT NULL,
            dag_version TEXT NOT NULL,
```

```
            exec_date   TEXT NOT NULL,
            row_count   INTEGER NOT NULL,
            status      TEXT NOT NULL,
            logged_at   TEXT NOT NULL
        )
    ''')
    conn.commit(); conn.close()
def drop_day(date_str: str):
    """
    Remove all rows for a given date from both staging and report tables.
    Called at the start of each run to ensure full idempotency.
    """
    conn = get_conn(); cur = conn.cursor()
    cur.execute('DELETE FROM raw_orders    WHERE order_date = ?', (date_str,))
    cur.execute('DELETE FROM daily_revenue WHERE report_date = ?', (date_str,))
    conn.commit(); conn.close()
```

File 2 — include/data_gen.py

Generates synthetic order records for any given date. The same seed is used per date so backfill re-runs produce identical data.

```
# include/data_gen.py
"""
data_gen.py — Deterministic synthetic order generator.
Same date + same seed = same orders every time (safe for backfill).
"""
import random, uuid
from datetime import date
CATEGORIES  = ['Electronics', 'Clothing', 'Groceries']
PRICE_RANGE = {
    'Electronics': (500,  15000),
    'Clothing':    (200,  3000),
    'Groceries':   (50,   800),
}
def generate_orders(for_date: date, n_orders: int = 120) -> list[dict]:
    """
    Return a list of order dicts for for_date.
    Seed = YYYYMMDD integer — deterministic across backfills.
    """
    seed = int(for_date.strftime('%Y%m%d'))
    rng  = random.Random(seed)
    orders = []
    for _ in range(n_orders):
        cat    = rng.choice(CATEGORIES)
        lo, hi = PRICE_RANGE[cat]
        orders.append({
            'order_id':    str(uuid.UUID(int=rng.getrandbits(128))),
            'order_date':  for_date.isoformat(),
            'category':    cat,
            'amount':      round(rng.uniform(lo, hi), 2),
            'customer_id': f'CUST{rng.randint(1000, 9999)}',
```

```
        })
    return orders
```

File 3 — dags/retailmart_revenue_v1.py (The Main DAG)

This is the **primary learning vehicle**. Every backfill, catchup, and versioning exercise operates on this DAG. Read every comment — they explain the Airflow concept being demonstrated by that line of code.

> **★ IMPORTANT**
>
> This DAG uses start_date=datetime(2026, 1, 1) deliberately. When you run a backfill from Jan to Apr you are filling four months of simulated missing data.

```
# dags/retailmart_revenue_v1.py
"""
retailmart_revenue_v1.py
RetailMart Daily Revenue Pipeline — Version 1
Pipeline:
  generate_orders
       |
  validate_orders
       |
  load_to_staging
       |
  compute_revenue
       |
  load_to_report
       |
  audit_log
"""
import sys, os
sys.path.insert(0, '/usr/local/airflow/include')   # Astro include path
import sqlite3
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
# ■■ project imports ■■
from db_utils  import init_db, drop_day, get_conn, DB_PATH
from data_gen  import generate_orders
DAG_VERSION = 'v1'
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# TASK FUNCTIONS
# Each function receives **context injected by Airflow.
# ALWAYS use context['data_interval_start'] for the date —
# this is what makes backfill work correctly.
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def task_generate_orders(**context):
    """Generate synthetic orders for this run's data interval date."""
    init_db()   # Creates tables if they don't exist yet
    # data_interval_start is the logical date for this run.
    # During backfill it will be Jan 1, Jan 2 ... not today.
    run_date = context['data_interval_start'].date()
    print(f'[generate] Generating orders for {run_date}')
    orders = generate_orders(for_date=run_date, n_orders=120)
    # Push the order list to XCom so downstream tasks can use it
```

```
    context['ti'].xcom_push(key='orders',    value=orders)
    context['ti'].xcom_push(key='run_date',  value=str(run_date))
    print(f'[generate] {len(orders)} orders pushed to XCom')

def task_validate_orders(**context):
    """
    Validate order quality. Raises ValueError on bad data.
    You can inject failure here to practise --rerun-failed-tasks.
    """
    orders = context['ti'].xcom_pull(task_ids='generate_orders', key='orders')
    errors = []
    for o in orders:
        if o['amount'] <= 0:
            errors.append(f"order {o['order_id']}: non-positive amount")
        if o['category'] not in ('Electronics', 'Clothing', 'Groceries'):
            errors.append(f"order {o['order_id']}: unknown category")
        if not o['customer_id']:
            errors.append(f"order {o['order_id']}: missing customer_id")
    if errors:
        raise ValueError(f'Validation failed:\n' + '\n'.join(errors))
    print(f'[validate] All {len(orders)} orders passed validation')

def task_load_to_staging(**context):
    """
    Load validated orders into raw_orders staging table.
    Drops the day first — idempotent (safe to re-run).
    """
    ti       = context['ti']
    orders   = ti.xcom_pull(task_ids='generate_orders', key='orders')
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    # IDEMPOTENCY: delete existing rows for this date before inserting
    drop_day(run_date)
    print(f'[staging] Cleared existing rows for {run_date}')
    conn = get_conn()
    conn.executemany(
        'INSERT INTO raw_orders VALUES (:order_id,:order_date,:category,:amount,:customer_id
)',
        orders,
    )
    conn.commit(); conn.close()
    print(f'[staging] Inserted {len(orders)} rows into raw_orders')

def task_compute_revenue(**context):
    """Aggregate revenue by category from staging table."""
    run_date = context['ti'].xcom_pull(task_ids='generate_orders', key='run_date')
    conn = get_conn(); cur = conn.cursor()
    cur.execute('''
        SELECT category,
               ROUND(SUM(amount), 2) AS total_rev,
               COUNT(*)              AS order_count
        FROM raw_orders
        WHERE order_date = ?
        GROUP BY category
    ''', (run_date,))
    rows = cur.fetchall(); conn.close()
    revenue = [{'category': r[0], 'total_rev': r[1], 'order_count': r[2]}
```

```
            for r in rows]
    context['ti'].xcom_push(key='revenue', value=revenue)
    print(f'[compute] Revenue computed for {run_date}: {revenue}')

def task_load_to_report(**context):
    """
    Upsert daily revenue into daily_revenue reporting table.
    INSERT OR REPLACE ensures idempotency — same date can be loaded many times.
    """
    ti       = context['ti']
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    revenue  = ti.xcom_pull(task_ids='compute_revenue',  key='revenue')
    now      = datetime.utcnow().isoformat()

    conn = get_conn()
    for r in revenue:
        conn.execute('''
            INSERT OR REPLACE INTO daily_revenue
                (report_date, category, total_rev, order_count, dag_version, loaded_at)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', (run_date, r['category'], r['total_rev'],
              r['order_count'], DAG_VERSION, now))
    conn.commit(); conn.close()
    print(f'[report] {len(revenue)} category rows loaded for {run_date} [{DAG_VERSION}]')

def task_audit_log(**context):
    """Write a run audit record for observability."""
    ti       = context['ti']
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    revenue  = ti.xcom_pull(task_ids='compute_revenue',  key='revenue')
    total_rows = sum(r['order_count'] for r in revenue)

    conn = get_conn()
    conn.execute('''
        INSERT OR REPLACE INTO pipeline_audit
            (run_id, dag_id, dag_version, exec_date, row_count, status, logged_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (context['run_id'], context['dag'].dag_id, DAG_VERSION,
          run_date, total_rows, 'success', datetime.utcnow().isoformat()))
    conn.commit(); conn.close()
    print(f'[audit] Run {context["run_id"]} logged: {total_rows} orders')

# ═══════════════════════════════════════════════════════════════════════
# DAG DEFINITION
# start_date = 2026-01-01 intentionally — backfill gap exists from here.
# catchup    = False initially — you will change this in the lab.
# ═══════════════════════════════════════════════════════════════════════
with DAG(
    dag_id='retailmart_revenue_v1',
    description='Daily revenue pipeline for RetailMart — Version 1',
    start_date=datetime(2026, 1, 1),
    schedule_interval='@daily',
    catchup=False,             # LAB EXERCISE: change to True and observe
    max_active_runs=3,         # Limits parallel backfill concurrency
    default_args={
        'owner':        'data-team',
        'retries':      2,
        'retry_delay':  timedelta(minutes=3),
```

```
        'email_on_failure': False,
    },
    tags=['retailmart', 'etl', 'revenue', DAG_VERSION],
    doc_md='''
## RetailMart Daily Revenue Pipeline v1
Generates synthetic orders, validates, stages, aggregates,
and loads daily revenue summaries into SQLite.
Safe for backfill — all tasks are idempotent.
    ''',
) as dag:
    t1 = PythonOperator(task_id='generate_orders',  python_callable=task_generate_orders,  p
rovide_context=True)
    t2 = PythonOperator(task_id='validate_orders',  python_callable=task_validate_orders,  p
rovide_context=True)
    t3 = PythonOperator(task_id='load_to_staging',  python_callable=task_load_to_staging,  p
rovide_context=True)
    t4 = PythonOperator(task_id='compute_revenue',  python_callable=task_compute_revenue,  p
rovide_context=True)
    t5 = PythonOperator(task_id='load_to_report',   python_callable=task_load_to_report,   p
rovide_context=True)
    t6 = PythonOperator(task_id='audit_log',        python_callable=task_audit_log,        p
rovide_context=True)
    t1 >> t2 >> t3 >> t4 >> t5 >> t6
```

File 4 — dags/retailmart_revenue_v2.py (Versioned Upgrade)

Version 2 adds a **discount column** to the revenue report and renames a task. This is the file you deploy to demonstrate DAG versioning — v1 keeps running for historical dates while v2 handles dates from May 2026 onward.

```
# dags/retailmart_revenue_v2.py
"""
retailmart_revenue_v2.py
RetailMart Daily Revenue Pipeline — Version 2
Changes from v1:
  - Added discount_total column to daily_revenue (schema migration task)
  - Task 'load_to_staging' renamed to 'ingest_to_staging' (shows version break)
  - start_date = 2026-05-01  (takes over from v1 which ends 2026-04-30)
"""
import sys
sys.path.insert(0, '/usr/local/airflow/include')
import sqlite3
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from db_utils import init_db, drop_day, get_conn
from data_gen import generate_orders
DAG_VERSION = 'v2'
def task_migrate_schema(**context):
    """
    NEW in v2: Add discount_total column if it doesn't exist.
    Uses ALTER TABLE ... (SQLite ignores if column exists via try/except).
    """
    init_db()
    conn = get_conn()
    try:
        conn.execute('ALTER TABLE daily_revenue ADD COLUMN discount_total REAL DEFAULT 0.0')
        conn.commit()
        print('[migrate] Added discount_total column to daily_revenue')
    except Exception:
        print('[migrate] Column already exists — skipping')
    conn.close()
def task_generate_orders(**context):
    init_db()
    run_date = context['data_interval_start'].date()
    orders = generate_orders(for_date=run_date, n_orders=150)  # v2: 150 orders/day
    context['ti'].xcom_push(key='orders',   value=orders)
    context['ti'].xcom_push(key='run_date', value=str(run_date))
    print(f'[v2 generate] {len(orders)} orders for {run_date}')
def task_validate_orders(**context):
    orders = context['ti'].xcom_pull(task_ids='generate_orders', key='orders')
    bad = [o for o in orders if o['amount'] <= 0 or not o['category']]
    if bad:
        raise ValueError(f'{len(bad)} invalid orders detected')
    print(f'[v2 validate] {len(orders)} orders valid')
def task_ingest_to_staging(**context):   # RENAMED from load_to_staging in v1
```

```
    ti       = context['ti']
    orders   = ti.xcom_pull(task_ids='generate_orders', key='orders')
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    drop_day(run_date)
    conn = get_conn()
    conn.executemany(
        'INSERT INTO raw_orders VALUES (:order_id,:order_date,:category,:amount,:customer_id
)',
        orders,
    )
    conn.commit(); conn.close()
    print(f'[v2 ingest] {len(orders)} rows staged')
def task_compute_revenue(**context):
    run_date = context['ti'].xcom_pull(task_ids='generate_orders', key='run_date')
    conn = get_conn(); cur = conn.cursor()
    cur.execute('''
        SELECT category,
               ROUND(SUM(amount), 2)         AS total_rev,
               COUNT(*)                       AS order_count,
               ROUND(SUM(amount) * 0.05, 2)  AS discount_total
        FROM raw_orders WHERE order_date = ?
        GROUP BY category
    ''', (run_date,))
    rows = cur.fetchall(); conn.close()
    revenue = [{'category': r[0], 'total_rev': r[1],
                'order_count': r[2], 'discount_total': r[3]} for r in rows]
    context['ti'].xcom_push(key='revenue', value=revenue)
    print(f'[v2 compute] {revenue}')
def task_load_to_report(**context):
    ti       = context['ti']
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    revenue  = ti.xcom_pull(task_ids='compute_revenue',  key='revenue')
    now      = datetime.utcnow().isoformat()
    conn = get_conn()
    for r in revenue:
        conn.execute('''
            INSERT OR REPLACE INTO daily_revenue
                (report_date, category, total_rev, order_count,
                 discount_total, dag_version, loaded_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (run_date, r['category'], r['total_rev'],
              r['order_count'], r.get('discount_total', 0.0),
              DAG_VERSION, now))
    conn.commit(); conn.close()
    print(f'[v2 report] {len(revenue)} rows loaded [{DAG_VERSION}]')
def task_audit_log(**context):
    ti       = context['ti']
    run_date = ti.xcom_pull(task_ids='generate_orders', key='run_date')
    revenue  = ti.xcom_pull(task_ids='compute_revenue',  key='revenue')
    total_rows = sum(r['order_count'] for r in revenue)
    conn = get_conn()
    conn.execute('''
        INSERT OR REPLACE INTO pipeline_audit
```

```
            (run_id, dag_id, dag_version, exec_date, row_count, status, logged_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (context['run_id'], context['dag'].dag_id, DAG_VERSION,
          run_date, total_rows, 'success', datetime.utcnow().isoformat()))
    conn.commit(); conn.close()

with DAG(
    dag_id='retailmart_revenue_v2',
    description='Daily revenue pipeline — Version 2 (adds discount_total)',
    start_date=datetime(2026, 5, 1),   # Takes over from v1
    schedule_interval='@daily',
    catchup=False,
    max_active_runs=3,
    default_args={
        'owner': 'data-team',
        'retries': 2,
        'retry_delay': timedelta(minutes=3),
        'email_on_failure': False,
    },
    tags=['retailmart', 'etl', 'revenue', DAG_VERSION],
) as dag:
    t0 = PythonOperator(task_id='migrate_schema',     python_callable=task_migrate_schema,
    provide_context=True)
    t1 = PythonOperator(task_id='generate_orders',    python_callable=task_generate_orders,
    provide_context=True)
    t2 = PythonOperator(task_id='validate_orders',    python_callable=task_validate_orders,
    provide_context=True)
    t3 = PythonOperator(task_id='ingest_to_staging',  python_callable=task_ingest_to_staging
,   provide_context=True)
    t4 = PythonOperator(task_id='compute_revenue',    python_callable=task_compute_revenue,
    provide_context=True)
    t5 = PythonOperator(task_id='load_to_report',     python_callable=task_load_to_report,
    provide_context=True)
    t6 = PythonOperator(task_id='audit_log',          python_callable=task_audit_log,
    provide_context=True)
    t0 >> t1 >> t2 >> t3 >> t4 >> t5 >> t6
```

File 5 — dags/catchup_compare/catchup_true_demo.py

A lightweight DAG with catchup=True. When enabled for the first time alongside catchup_false_demo, the Grid view contrast is immediately visible.

```
# dags/catchup_compare/catchup_true_demo.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
with DAG(
    dag_id='catchup_true_demo',
    start_date=datetime(2026, 4, 1),   # 40 days ago
    schedule_interval='@daily',
    catchup=True,                      # Creates ~40 runs on first enable
    max_active_runs=5,
    tags=['demo', 'catchup', 'compare'],
) as dag:
    def show_interval(**ctx):
        start = ctx['data_interval_start']
        end   = ctx['data_interval_end']
        print(f'[catchup=True]  interval: {start.date()} -> {end.date()}')
    PythonOperator(
        task_id='show_interval',
        python_callable=show_interval,
        provide_context=True,
    )
```

File 6 — dags/catchup_compare/catchup_false_demo.py

```
# dags/catchup_compare/catchup_false_demo.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
with DAG(
    dag_id='catchup_false_demo',
    start_date=datetime(2026, 4, 1),   # Same start_date as above
    schedule_interval='@daily',
    catchup=False,                     # Only 1 run created on enable
    tags=['demo', 'catchup', 'compare'],
) as dag:
    def show_interval(**ctx):
        start = ctx['data_interval_start']
        end   = ctx['data_interval_end']
        print(f'[catchup=False] interval: {start.date()} -> {end.date()}')
    PythonOperator(
        task_id='show_interval',
        python_callable=show_interval,
        provide_context=True,
    )
```

File 7 — tests/test_retailmart_dag.py

Pytest tests that validate the DAG loads correctly, has no import errors, and the task dependency chain is intact. These run in CI before every deploy.

```
# tests/test_retailmart_dag.py
"""
DAG integrity tests — run with: astro dev pytest tests/
or locally: pytest tests/ -v
"""
import pytest
from airflow.models import DagBag

DAG_BAG = DagBag(dag_folder='dags/', include_examples=False)

def test_no_import_errors():
    """No DAG file should have import errors."""
    assert DAG_BAG.import_errors == {}, \
        f'Import errors: {DAG_BAG.import_errors}'

def test_v1_dag_exists():
    assert 'retailmart_revenue_v1' in DAG_BAG.dags

def test_v1_task_count():
    dag = DAG_BAG.dags['retailmart_revenue_v1']
    assert len(dag.tasks) == 6, f'Expected 6 tasks, got {len(dag.tasks)}'

def test_v1_task_order():
    dag = DAG_BAG.dags['retailmart_revenue_v1']
    task_ids = [t.task_id for t in dag.topological_sort()]
    expected = ['generate_orders', 'validate_orders', 'load_to_staging',
                'compute_revenue', 'load_to_report', 'audit_log']
    assert task_ids == expected, f'Order mismatch: {task_ids}'

def test_v1_catchup_is_false():
    """Default should be catchup=False to prevent accidental mass scheduling."""
    dag = DAG_BAG.dags['retailmart_revenue_v1']
    assert dag.catchup is False

def test_v2_dag_exists():
    """v2 should exist once retailmart_revenue_v2.py is added."""
    if 'retailmart_revenue_v2' in DAG_BAG.dags:
        dag = DAG_BAG.dags['retailmart_revenue_v2']
        assert len(dag.tasks) == 7, 'v2 should have 7 tasks (including migrate_schema)'

def test_catchup_demos_exist():
    for dag_id in ('catchup_true_demo', 'catchup_false_demo'):
        assert dag_id in DAG_BAG.dags, f'{dag_id} not found'
```

Step Lab Exercises

Run each lab in order — each one uses the DAG you just built

Lab 1 — Execute Backfill (Jan–Apr 2026 Missing Data)

RetailMart's pipeline started on May 1 2026. The analytics team has now discovered that order data exists in the source system from January 2026. You need to backfill four months of revenue data.

1. **Confirm v1 DAG is enabled (catchup=False)**

   Open http://localhost:8080, find retailmart_revenue_v1, confirm it is ON. The Grid should show only recent runs (if any).

2. **Run a dry-run to preview intervals**

   This shows which logical_dates would be created without actually running.

3. **Execute the backfill for Jan–Apr 2026**

   This triggers 120 DAG runs (31+28+31+30 days). max_active_runs=3 limits concurrency.

4. **Monitor progress in the Airflow UI**

   Grid view: watch the green squares fill in left-to-right. Each column is one day, each row is one task.

5. **Verify data in SQLite**

   Connect to the container and query the reporting table to confirm all dates loaded.

```
# Step 2 — Dry run
astro dev run dags backfill \
    -s 2026-01-01 -e 2026-05-01 \
    --dry-run \
    retailmart_revenue_v1
# Step 3 — Actual backfill
astro dev run dags backfill \
    -s 2026-01-01 \
    -e 2026-05-01 \
    retailmart_revenue_v1
# Step 5 — Verify in SQLite
astro dev bash   # Opens a shell inside the Airflow container
# Inside the container:
sqlite3 /tmp/retailmart.db
# SQLite queries to verify:
.mode column
.headers on
-- Check how many dates were loaded
```

```
SELECT COUNT(DISTINCT report_date) AS loaded_days FROM daily_revenue;
-- Expected: 120
-- Sample the first 5 dates
SELECT * FROM daily_revenue ORDER BY report_date LIMIT 15;
-- Audit log confirms backfill run IDs
SELECT dag_version, exec_date, row_count
FROM pipeline_audit
ORDER BY exec_date LIMIT 10;
.quit
```

> **✔ PRO TIP**
>
> Notice that run_id in pipeline_audit starts with backfill__2026-01-01... rather than scheduled__. This is how Airflow marks backfill-triggered runs.

Lab 2 — Compare Catchup Behaviour

Enable the two catchup demo DAGs at exactly the same moment and observe how differently Airflow handles the same 40-day gap.

1. **Ensure both demo DAGs are present and PAUSED**

   In the UI, confirm catchup_true_demo and catchup_false_demo are visible and paused.

2. **Enable BOTH at the same time**

   Toggle ON catchup_true_demo first, immediately toggle ON catchup_false_demo. Refresh the Grid view within 10 seconds.

3. **Observe the Grid view difference**

   catchup_true_demo: ~40 queued runs filling the entire historical grid. catchup_false_demo: only 1 run for today.

4. **Change v1 to catchup=True and restart**

   Edit retailmart_revenue_v1.py, change catchup=False to catchup=True, save. Restart astro dev restart. Pause then unpause the DAG — observe mass scheduling.

5. **Revert to catchup=False**

   Change back to catchup=False and restart. This is the safe production default.

```
# Edit v1 to flip catchup (Step 4)
# In retailmart_revenue_v1.py, change:
#   catchup=False
# to:
#   catchup=True
# Restart the environment to pick up the change
```

```
astro dev restart
# Pause the DAG (so it doesn't auto-run), then unpause
astro dev run dags pause   retailmart_revenue_v1
astro dev run dags unpause retailmart_revenue_v1
# Watch runs flood in — then immediately pause again to stop them
astro dev run dags pause retailmart_revenue_v1
# Count how many runs were triggered
astro dev run dags list-runs -d retailmart_revenue_v1 | wc -l
```

Lab 3 — Implement DAG Versioning (v1 → v2)

The business wants a discount_total column in the revenue report from May 2026 onward. You will deploy v2 without touching v1's historical data.

1. **Set v1 end_date to 2026-04-30**

   Open retailmart_revenue_v1.py, add end_date=datetime(2026, 4, 30) to the DAG. This stops v1 from creating any new runs after April.

2. **Place retailmart_revenue_v2.py in dags/**

   The file already uses start_date=datetime(2026, 5, 1). Astro will auto-detect it within 30 seconds.

3. **Confirm both DAGs are visible in the UI**

   You should see retailmart_revenue_v1 (historical) and retailmart_revenue_v2 (current). v1 shows no new runs scheduled. v2 shows today's run.

4. **Trigger v2 manually for 2026-05-01**

   Click Trigger DAG in the UI, set logical_date to 2026-05-01. After success, check the daily_revenue table — the new row should have dag_version=v2.

5. **Query audit log to see both versions**

   The pipeline_audit table will show v1 rows for Jan–Apr and v2 rows for May onward.

```
# Step 1 — add end_date to v1 (inside the DAG() block):
#   end_date=datetime(2026, 4, 30),
# Step 4 — verify the schema migration happened
astro dev bash
sqlite3 /tmp/retailmart.db
-- Check columns in daily_revenue
PRAGMA table_info(daily_revenue);
-- Should show discount_total column added by migrate_schema task
-- Compare v1 vs v2 rows
SELECT dag_version, MIN(report_date), MAX(report_date), COUNT(*)
FROM daily_revenue
GROUP BY dag_version;
```

```
-- audit log cross-version view
SELECT dag_version, exec_date, row_count
FROM pipeline_audit
ORDER BY exec_date DESC LIMIT 10;
```

Lab 4 — Promote DAG Across Environments

Push retailmart_revenue_v2.py through the Dev → Stage → Production pipeline using the GitHub Actions workflow and your active Astro deployment.

1

Initialise Git repo and push to GitHub

Commit all project files to a GitHub repository. Create branches: develop, staging, and main.

2

Add GitHub secrets for Astro tokens

In GitHub repo Settings > Secrets: add ASTRONOMER_KEY_ID, ASTRONOMER_KEY_SECRET, and your DEV/STG/PRD deployment IDs.

3

Push to develop → triggers Dev deploy

git checkout develop && git push. GitHub Actions runs pytest then deploys to Dev deployment automatically.

4

Open PR: develop → staging

Create a PR and get it reviewed. Merge triggers Stage deploy with 1 approval gate.

5

Tag a release → triggers Production deploy

git tag v2.0.0 && git push origin v2.0.0. Production job pauses for 2 reviewer approvals before executing.

```
# Step 1 — initialise git
cd retailmart-pipeline
git init
git remote add origin https://github.com/<your-org>/retailmart-pipeline.git
git checkout -b develop
git add .
git commit -m 'feat: retailmart revenue pipeline v1 + v2'
git push -u origin develop
# Step 3 — watch CI run
#   GitHub Actions tab > workflow run > pytest logs should show 7 tests passed
# Step 5 — tag for production
git checkout main
git merge staging
git tag -a v2.0.0 -m 'retailmart_revenue_v2: discount_total column'
git push origin v2.0.0
# Verify the deploy reached Astro
astro deployment inspect $PRD_DEPLOYMENT_ID
```

```
astro deployment logs    $PRD_DEPLOYMENT_ID --scheduler | tail -20
```

on Queries & Expected Outputs

Use these SQLite queries to confirm each lab completed correctly

Lab Check	Query / Action	Expected Result
Lab 1 — Backfill complete	SELECT COUNT(DISTINCT report_date) FROM daily_revenue;	120 (one per day Jan–Apr 2026)
Lab 1 — All 3 categories loaded	SELECT category, COUNT(*) FROM daily_revenue GROUP BY category;	Electronics / Clothing / Groceries, each 120 rows
Lab 1 — Backfill run IDs	SELECT run_id FROM pipeline_audit LIMIT 3;	backfill__2026-01-01T00:00:00+00:00
Lab 2 — Catchup run count	SELECT COUNT(*) FROM pipeline_audit WHERE dag_id='catchup_true_demo';	~40 rows
Lab 3 — Both versions present	SELECT DISTINCT dag_version FROM daily_revenue;	v1, v2
Lab 3 — Schema migration	PRAGMA table_info(daily_revenue);	discount_total column visible
Lab 4 — v2 in Production	Astro UI > Deployment > DAGs tab	retailmart_revenue_v2 listed and active

✔ PRO TIP

If any query returns unexpected results, re-check that data_interval_start is used (not datetime.now()), and that drop_day() runs before inserts in load_to_staging. These two patterns are the foundation of all backfill safety.

Common Errors & Fixes

Error	Fix
ModuleNotFoundError: db_utils	sys.path.insert(0, '/usr/local/airflow/include') missing at top of DAG file.
sqlite3.OperationalError: no such table	init_db() not called in generate_orders task. Ensure it is the first call.
XCom key 'orders' returned None	Task ID mismatch in xcom_pull — check task_ids parameter matches exactly.
Backfill creates 0 runs	start_date is in the future, or the date range is outside start_date/end_date window.
Duplicate rows in daily_revenue	Not using INSERT OR REPLACE — check load_to_report task SQL.
v2 tasks not visible in UI	File not saved to dags/ or Airflow hasn't refreshed. Wait 30s or restart dev env.